



MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 2

Filter & Smoothing Time Series Data

Abstrak

Pertemuan ini membahas teknik filtering dan smoothing untuk data time series sensor industri yang selalu tercampur

Sub - CPMK

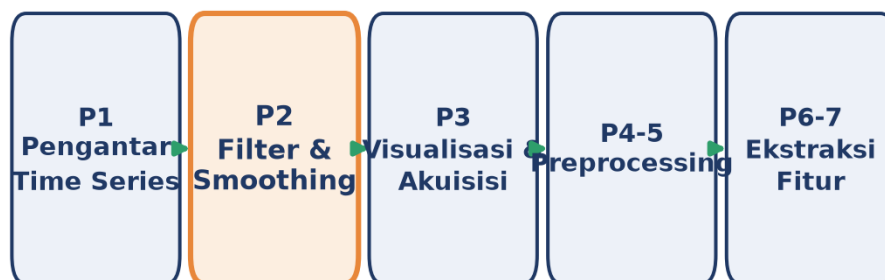
Sub-CPMK 1.2 – Mampu menganalisis kebutuhan sistem untuk efisiensi dan keberlanjutan

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

🔗 **Modul Interaktif:** <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-2.html>. Pada layar masuk, pilih tombol “👁️ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Pertemuan kedua ini melanjutkan pemahaman dasar time series dari Pertemuan 1 menuju keterampilan praktis pertama: membersihkan sinyal sensor dari noise sebelum dianalisis lebih lanjut. Setiap pengukuran sensor industri (getaran, suhu, arus, tekanan) selalu tercampur noise dan spike sporadis. Modul ini membahas empat teknik filter utama yaitu Moving Average, Exponential Moving Average (EMA), Savitzky-Golay, dan median filter, beserta trade-off masing-masing dan bagaimana menyusun pipeline denoising yang tidak merusak kandungan sinyal asli. Gambar 1 menunjukkan posisi Pertemuan 2 dalam keseluruhan alur mata kuliah.



Gambar 1. Posisi Pertemuan 2 (Filter & Smoothing) dalam alur mata kuliah Optimalisasi & Otomasi.

Materi mencakup teori sumber-sumber noise pada sensor industri, empat teknik filter time-domain beserta parameter dan trade-off-nya, pemilihan filter berdasarkan tipe noise, serta pipeline rekomendasi untuk vibration monitoring. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan latihan komputasi.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 1.2:** Mampu menganalisis kebutuhan sistem untuk efisiensi dan keberlanjutan, yaitu memilih teknik filter yang tepat berdasarkan trade-off komputasi, lag, dan preservasi sinyal untuk sistem monitoring yang berkelanjutan (CPMK 1).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan sumber-sumber noise pada sensor industri; menerapkan Moving Average dan EMA beserta pemilihan parameternya; menerapkan Savitzky-Golay dan median filter untuk preservasi

puncak dan penghapusan spike; serta menyusun pipeline filter yang tepat untuk vibration monitoring.

NOISE DALAM TIME SERIES

Setiap pengukuran sensor mengandung noise: model sinyal teramati adalah $x_{\text{obs}}(t) = s(t) + n(t)$, yaitu sinyal asli $s(t)$ ditambah noise $n(t)$. Sumber noise pada sensor industri beragam: thermal noise (Johnson noise) dari komponen elektronik, quantization noise akibat resolusi ADC terbatas, electromagnetic interference (EMI) dari motor dan inverter di sekitar sensor, mechanical noise dari getaran struktur, dan environmental drift akibat perubahan suhu lingkungan.

$$SNR = P_{\text{sinyal}} / P_{\text{noise}}$$

Trade-off smoothing: Tanpa filter, noise menimbulkan konsekuensi nyata: RMS bias ke atas (nilai RMS jadi lebih besar dari nilai sebenarnya), kesalahan deteksi peak, spektrum FFT menjadi berisik, dan autokorelasi menurun. Namun smoothing yang berlebihan sama berbahayanya dengan tidak sama sekali: kurang smoothing menyisakan noise yang mengaburkan analisis, sementara smoothing berlebihan justru menghapus kandungan sinyal yang bermakna (mis. impulsive event tanda kerusakan dini bearing). Kunci keseimbangan ini terletak pada pemilihan window size, $w_{\text{optimal}} = f(f_{\text{sinyal}}, f_{\text{noise}})$.

- **Causal (trailing):** filter hanya memakai data masa lalu (window trailing), cocok untuk real-time monitoring karena bisa dihitung saat data baru masuk, tetapi punya delay/lag terhadap sinyal asli.
- **Non-causal (centered):** filter memakai data masa lalu dan masa depan sekaligus (window centered), menghasilkan zero phase shift (tanpa lag) tetapi hanya bisa dihitung offline karena butuh data masa depan yang belum tersedia saat real-time.

MOVING AVERAGE

Moving Average (MA) adalah filter smoothing paling fundamental: merata-ratakan k titik data di sekitar setiap sampel. Ada dua varian utama tergantung posisi window terhadap titik saat ini. Hubungan ini dirangkum dalam Persamaan (1).

$$\text{Centered: } y'_t = (1/k) \cdot \sum_{i=-k/2..k/2} x_{t+i}, \quad \text{Trailing: } y'_t = (1/k) \cdot \sum_{i=0..k-1} x_{t-i} \quad (1)$$

- **MA Centered:** rata-rata simetris kiri-kanan titik t ; zero phase shift tetapi butuh data masa depan sehingga hanya cocok analisis offline.

- **MA Trailing:** rata-rata hanya dari titik t ke belakang; causal dan real-time-ready, tetapi menimbulkan lag $\approx (k-1)/2$ sampel terhadap sinyal asli.
- **Pemilihan window:** window k dipilih mengikuti rasio sampling rate terhadap frekuensi cutoff yang diinginkan, $k \approx f_s/f_{\text{cutoff}}$. Window terlalu kecil menyisakan noise; window terlalu besar menumpulkan puncak dan menambah lag.

Kelemahan MA: bobot uniform pada seluruh titik window membuat frequency response-nya berbentuk sinc dengan sidelobe yang bocor (tidak benar-benar memotong frekuensi tinggi secara bersih), muncul boundary effect (NaN atau distorsi di tepi data), dan pada varian trailing menimbulkan lag. Implementasi praktis: `np.convolve(x, np.ones(k)/k, mode='valid')` atau `pd.Series(x).rolling(k).mean()`. Trik praktis: pakai trailing MA $k=10-20$ sebagai baseline bergerak, lalu tandai deviasi lebih dari 3σ sebagai kandidat anomali untuk diinvestigasi.

Tabel 1 merangkum pemilihan window k terhadap sampling rate dan frekuensi cutoff efektif yang dihasilkan, beserta lag yang ditimbulkan pada varian trailing.

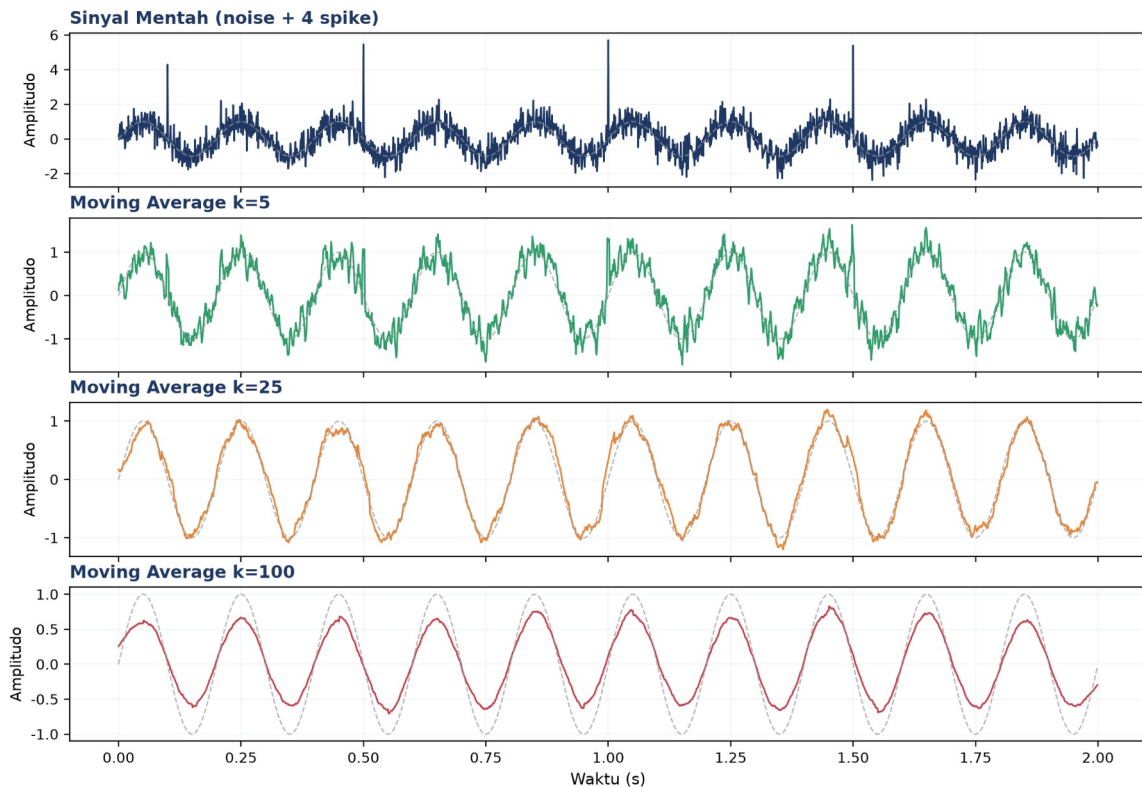
Tabel 1. Pemilihan window Moving Average terhadap sampling rate dan frekuensi cutoff efektif.

Sampling Rate	Window k	fcutoff Efektif	Lag (ms)
1000 Hz	5	~200 Hz	2 ms
1000 Hz	25	~40 Hz	12 ms
100 Hz	10	~10 Hz	50 ms
10 Hz	10	~1 Hz	500 ms
1 Hz	10	~0.1 Hz	5000 ms

Contoh Konkret: Pemilihan Window pada Deteksi Bearing Fault. Sebagai ilustrasi konkret pemilihan window, perhatikan accelerometer pada bearing motor 1500 rpm ($f_s=1000$ Hz) yang harus mendeteksi indikasi awal spalling pada frekuensi karakteristik bearing (BPFO) sekitar 120 Hz. Bila window k dipilih terlalu besar ($k=100$, fcutoff efektif ~10 Hz menurut Tabel 1), komponen 120 Hz justru ikut teredam habis sehingga sinyal kerusakan hilang dari hasil filter, kesalahan fatal untuk aplikasi predictive maintenance. Sebaliknya window $k=5$ (fcutoff efektif ~200 Hz) mempertahankan komponen 120 Hz tetapi hanya sedikit meredam noise broadband di sekitarnya. Pelajaran praktisnya: window Moving Average wajib dipilih berdasarkan frekuensi terendah yang ingin dipertahankan dalam analisis lanjutan, bukan sekadar mengejar tampilan sinyal yang tampak mulus secara visual.

Penanganan Boundary Effect: Boundary effect juga perlu ditangani secara eksplisit saat implementasi produksi. Pada mode `'valid'` (NumPy), output kehilangan $(k-1)$ titik di kedua ujung sinyal karena window tidak memiliki data tetangga yang cukup untuk dirata-ratakan;

pada mode ``same'', titik-titik tepi diisi dengan hasil zero-padding yang dapat mendistorsi nilai dekat awal maupun akhir rekaman, sehingga menyesatkan bila kebetulan kejadian penting (mis. start-up mesin) berada tepat di tepi buffer data. Praktik yang disarankan untuk sistem streaming: rekam buffer tambahan sepanjang minimal $k/2$ sampel di kedua sisi window analisis sebelum menerapkan filter, lalu buang buffer tersebut setelah proses filtering selesai, sehingga seluruh titik yang dilaporkan ke sistem monitoring benar-benar bebas dari distorsi tepi.



Gambar 2. Sinyal noisy (noise + 4 spike) dan hasil Moving Average pada tiga level window $k=5$, 25, dan 100.

Gambar 2 memperlihatkan efek window size secara langsung: $k=5$ hanya sedikit meredam noise dan spike masih tampak jelas; $k=25$ menghasilkan kompromi yang baik antara smoothing dan preservasi bentuk gelombang; sedangkan $k=100$ sudah terlalu agresif, mengaburkan amplitudo asli sinyal sinusoidal 5 Hz.

EXPONENTIAL MOVING AVERAGE (EMA)

EMA menghitung rata-rata bergerak secara rekursif, memberi bobot lebih besar pada observasi terbaru dan bobot yang meluruh secara eksponensial untuk observasi lama. Bentuk ringkasnya dituliskan pada Persamaan (2).

$$EMA_t = \alpha \cdot x_t + (1-\alpha) \cdot EMA_{t-1}, \quad \alpha \in (0, 1] \quad (2)$$

Bobot pengamatan ke- i dari belakang adalah $w_i = \alpha \cdot (1-\alpha)^i$, sehingga effective window (jumlah sampel yang secara efektif berkontribusi) kira-kira $N \approx 2/\alpha - 1$. Pemilihan α : α kecil (mis. 0,05) menghasilkan smoothing agresif dengan lag besar; α besar (mis. 0,5) menghasilkan filter yang responsif tetapi kurang meredam noise. Aturan praktis untuk menyamakan EMA dengan MA window- N adalah $\alpha \approx 2/(N+1)$.

- **Inisialisasi:** $EMA_0 = x_0$ (paling umum), $EMA_0 = \text{mean}(x[:k])$ untuk mengurangi bias awal, atau memakai periode warm-up sebelum EMA dipakai untuk keputusan.
- **Implementasi pandas:** `pd.Series(x).ewm(alpha=α, adjust=False).mean()`, perhatikan parameter `adjust=False` wajib dipakai untuk mendapatkan bentuk rekursif standar; `adjust=True` (default pandas) akan menghitung ulang bobot dari seluruh history, bukan formula rekursif yang dimaksud.
- **EMA vs MA:** EMA punya lag lebih kecil dan tidak perlu menyimpan seluruh window (hanya nilai EMA sebelumnya), sementara MA punya frequency response yang lebih bersih (less sidelobe leakage). EWMA control chart ($|x - EMA| > 3 \cdot \sqrt{(EMA_{var})}$) adalah teknik standar Statistical Process Control (SPC)/Six Σ untuk deteksi pergeseran baseline.

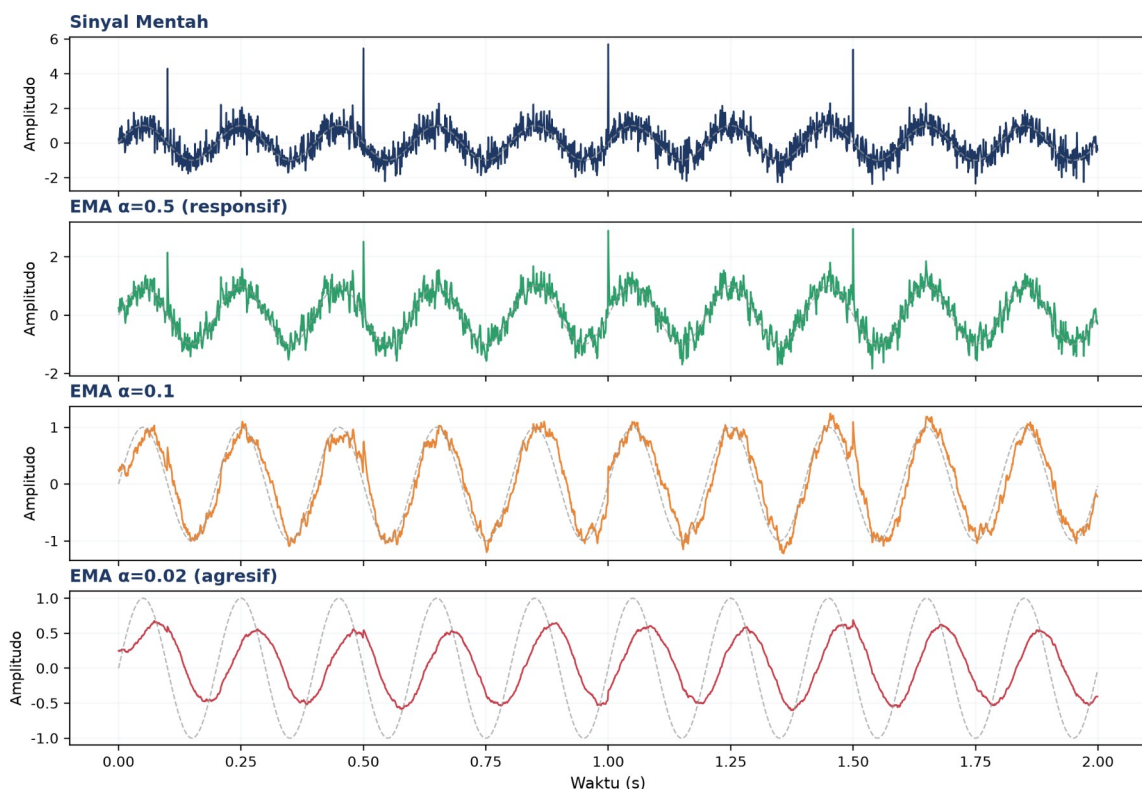
Tabel 2 merangkum karakter EMA pada beberapa nilai α beserta effective window dan use case yang sesuai.

Tabel 2. Pemilihan α EMA terhadap effective window dan karakter hasil filter.

α	Effective Window N	Karakter	Use Case
0,5	3	Sangat responsif, noise ikut lolos	Deteksi event cepat
0,3	5-6	Responsif, smoothing sedang	Alert real-time
0,1	19	Seimbang	Baseline umum
0,05	39	Smoothing kuat	Tren jangka menengah
0,01	199	Hampir flat	Baseline drift jangka panjang

Efisiensi Memori pada Sistem Monitoring Skala Besar: EMA banyak dipakai sebagai baseline adaptif pada sistem SCADA karena sifat rekursifnya sangat hemat memori: hanya satu nilai (EMA sebelumnya) yang perlu disimpan per channel sensor, berbeda dengan Moving Average yang butuh menyimpan seluruh k sampel terakhir dalam buffer. Untuk sistem monitoring dengan ratusan channel sensor berjalan pada embedded controller dengan RAM terbatas, perbedaan kebutuhan memori ini signifikan: MA $k=100$ pada 200 channel membutuhkan buffer 20.000 titik data, sedangkan EMA hanya membutuhkan 200 nilai skalar untuk mencapai smoothing setara.

Adaptive α : Pemilihan α juga dapat dilakukan secara adaptif (bukan konstan) berdasarkan kondisi operasi mesin: α diperbesar sementara saat start-up atau perubahan beban mendadak (agar EMA cepat mengejar perubahan level baseline yang legitimate), lalu dikembalikan ke nilai kecil semula saat kondisi steady-state tercapai. Teknik ini, dikenal sebagai variable-rate EMA atau adaptive exponential smoothing, mencegah trade-off klasik antara responsivitas dan stabilitas: pada α tetap kecil, EMA lambat mengikuti perubahan beban legitimate (false alarm delay); pada α tetap besar, EMA terlalu sensitif terhadap noise sesaat (false alarm rate tinggi).



Gambar 3. Sinyal noisy dan hasil EMA pada tiga nilai α : 0,5 (responsif), 0,1, dan 0,02 (agresif).

Perbandingan tiga level α : Terlihat pada Gambar 3 bahwa $\alpha=0,5$ nyaris tidak meredam noise (bahkan spike ikut lolos sebagian), $\alpha=0,1$ memberi smoothing yang seimbang, dan $\alpha=0,02$ menghasilkan kurva sangat halus namun dengan lag fase yang jelas terhadap sinyal asli garis putus-putus abu-abu.

SAVITZKY-GOLAY & MEDIAN FILTER

Savitzky-Golay (SG) bekerja dengan mem-fit polinomial orde p pada window $(2k+1)$ titik memakai least squares, lalu output-nya adalah nilai polinomial tersebut di titik pusat window; pendekatan ini menjaga bentuk puncak dan lembah jauh lebih baik dibanding MA. Median filter mengganti tiap titik dengan median dari window di sekitarnya, sangat efektif

untuk impulse noise/outlier karena sifatnya nonlinear (tidak ada sidelobe leakage seperti filter linear). Persamaan (3) memadatkan aturan tersebut.

$$\text{Median: } y^t = \text{median}(x[t-k : t+k+1]) \quad (3)$$

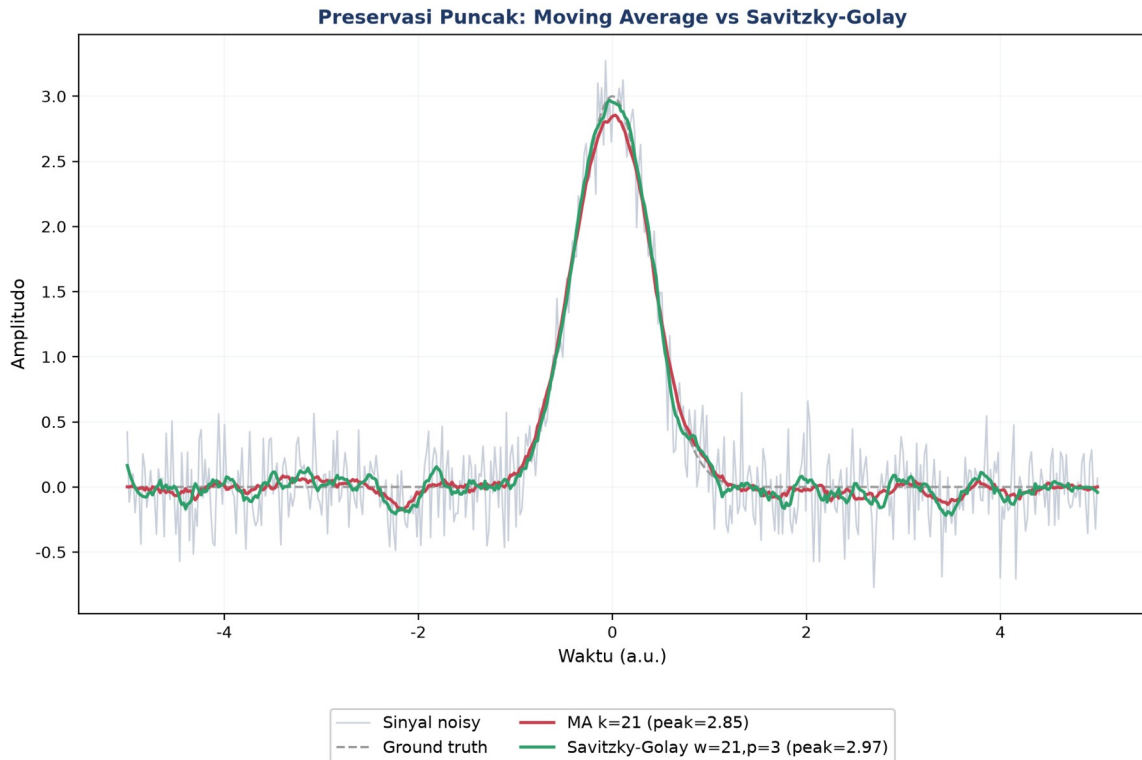
- **Parameter SG:** window w harus ganjil (rentang umum 5-101), orde polinomial p umumnya 2 (jaga smoothness), 3 (jaga curvature), atau 4-5 (jaga inflection point); syarat $p < w$. Titik awal yang aman: $w=11$, $p=2$.
- **Kapan pakai apa:** MA cocok untuk noise Gaussian dan kesederhanaan implementasi; EMA cocok untuk kebutuhan real-time adaptif; SG cocok saat preservasi puncak krusial (spektroskopi, analisis getaran); Median cocok khusus untuk outlier/spike/salt-and-pepper noise.
- **Chaining filter:** median dahulu untuk menghapus spike, baru Savitzky-Golay untuk smoothing yang tetap menjaga bentuk puncak: kombinasi ini menghasilkan sinyal bersih tanpa kehilangan informasi penting.

Tabel 3 membandingkan keempat filter dari sisi tipe noise yang paling cocok ditangani, kemampuan preservasi puncak, dan cocok-tidaknya untuk real-time.

Tabel 3. Perbandingan empat teknik filter terhadap tipe noise, preservasi puncak, dan kecocokan real-time.

Filter	Tipe Noise Terbaik	Preserve Peak?	Real-time?	Komputasi
Moving Average	Gaussian, broadband	Tidak	Ya (trailing)	$O(N)$
EMA	Gaussian, drift lambat	Tidak	Ya	$O(N)$
Savitzky-Golay	Gaussian, perlu jaga puncak	Ya	Tidak (perlu window penuh)	$O(N \cdot w \cdot p)$
Median	Impulsive/spike/outlier	Sebagian	Ya	$O(N \cdot w \cdot \log w)$

Constraint Komputasi pada Embedded Controller: Pemilihan filter yang tepat berdasarkan Tabel 3 juga harus mempertimbangkan constraint komputasi pada embedded controller: Savitzky-Golay dengan kompleksitas $O(N \cdot w \cdot p)$ menjadi kandidat termahal terutama saat window w besar dan orde polinomial p tinggi, sehingga pada sistem edge computing dengan sumber daya terbatas (mis. microcontroller ARM Cortex-M4 tanpa FPU), penerapan SG real-time pada banyak channel sensor sekaligus dapat menjadi bottleneck komputasi. Solusi praktis: precompute koefisien Savitzky-Golay untuk kombinasi (w, p) yang sudah ditentukan sebagai lookup table konstan, sehingga operasi filtering di runtime tereduksi menjadi dot product sederhana antara window data dengan koefisien yang sudah dihitung sebelumnya, alih-alih menyelesaikan sistem persamaan least squares setiap kali filter dipanggil pada tiap sampel baru.

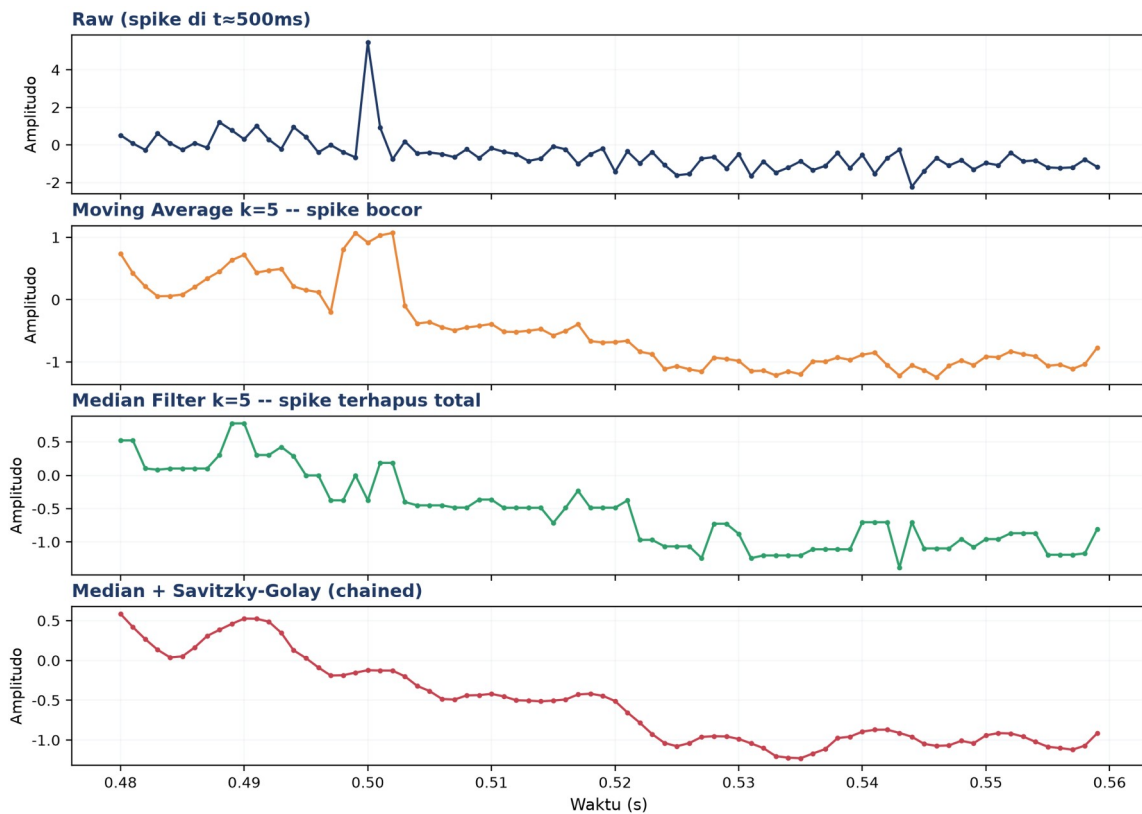


Gambar 4. Preservasi puncak pada sinyal Gaussian ber-noise: Moving Average $k=21$ vs Savitzky-Golay $w=21, p=3$.

Gambar 4 menunjukkan MA $k=21$ memangkas puncak sinyal dari nilai asli 3,0 menjadi hanya 2,85, sementara Savitzky-Golay $w=21, p=3$ mempertahankan puncak lebih dekat ke nilai asli (2,97), yaitu perbedaan krusial saat puncak tersebut merepresentasikan impulsive event nyata seperti impact awal kerusakan bearing, bukan sekadar noise.

Studi Kasus: Spike EMI pada Monitoring Bearing. Studi kasus nyata: pada monitoring bearing dengan sampling rate 1000 Hz, spike EMI berdurasi singkat (1-3 sampel, setara 1-3 milidetik) kerap muncul akibat starting arus motor tetangga pada instalasi kelistrikan yang sama. Bila spike ini tidak dihapus sebelum perhitungan RMS, nilai RMS getaran bisa melonjak signifikan sesaat dan memicu false alarm pada sistem monitoring kondisi, padahal kondisi mesin sesungguhnya normal. Median filter dengan window kecil ($w=3-5$) sangat efektif menghapus spike berdurasi pendek ini tanpa mendistorsi bentuk gelombang getaran periodik yang justru ingin dipertahankan untuk analisis lanjutan.

Peringatan Window Terlalu Besar: Sebaliknya, hati-hati menerapkan median filter dengan window terlalu besar ($w>15$) pada sinyal getaran yang mengandung impact event nyata (bukan spike noise), seperti benturan awal spalling bearing: window besar berisiko menghapus impact event legitimate tersebut karena dianggap outlier statistik, padahal justru itulah informasi diagnostik paling berharga yang sedang dicari oleh sistem predictive maintenance.

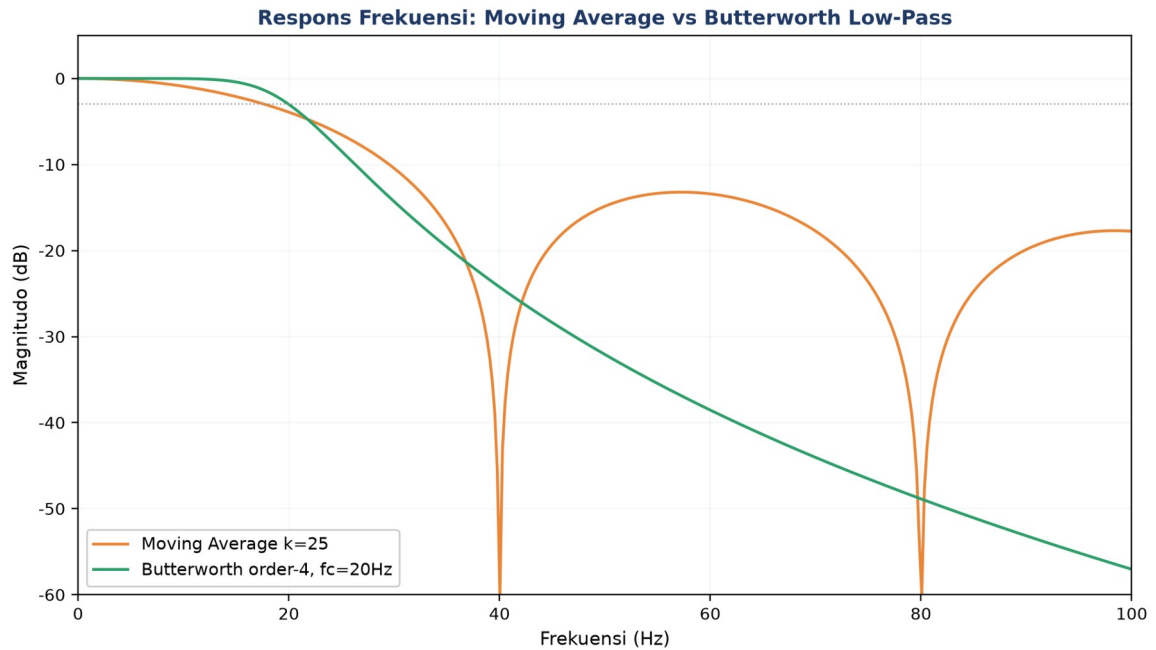


Gambar 5. Penghapusan spike: sinyal mentah, Moving Average $k=5$ (spike bocor), Median Filter $k=5$ (spike terhapus total), dan hasil chaining Median+Savitzky-Golay.

Linear vs nonlinear filtering: Perbandingan pada Gambar 5 memperjelas perbedaan fundamental filter linear vs nonlinear: MA $k=5$ hanya meredam sebagian amplitudo spike (spike bocor ke tetangganya), sementara median filter $k=5$ menghapus spike sepenuhnya dari sinyal. Panel terakhir menunjukkan hasil chaining median lalu Savitzky-Golay: sinyal bersih dari spike sekaligus halus tanpa distorsi bentuk gelombang.

Untuk memahami mengapa MA meninggalkan sidelobe (osilasi residual pada frekuensi tertentu) sementara filter Butterworth memberi roll-off yang lebih bersih, Gambar 6 membandingkan respons frekuensi keduanya secara eksplisit.

Implikasi pada Desain Anti-Aliasing: Perbedaan ini bukan sekadar teoretis: pada sistem anti-aliasing analog sebelum ADC, filter Butterworth (atau varian Chebyshev/Bessel) selalu dipilih dibanding pendekatan moving-average karena roll-off yang monoton menjamin komponen di atas Nyquist benar-benar teredam, sedangkan sidelobe MA berpotensi meloloskan sebagian energi frekuensi tinggi yang kemudian ter-alias kembali ke pita frekuensi rendah setelah sampling, mencemari hasil analisis FFT dengan komponen palsu yang tidak pernah benar-benar ada pada sinyal asli.



Gambar 6. Respons frekuensi Moving Average $k=25$ (sinc-like, sidelobe bocor) dibandingkan Butterworth low-pass order-4 $f_c=20\text{Hz}$ (roll-off bersih, monoton turun).

Moving Average menunjukkan pola sidelobe klasik filter sinc: magnitudo turun ke titik nol (null) di frekuensi tertentu (40 dan 80 Hz untuk $k=25$ pada $f_s=1000$ Hz) namun kembali naik di antaranya, artinya sebagian noise pada rentang frekuensi tersebut justru LOLOS tidak teredam. Butterworth order-4 menurun secara monoton dan konsisten setelah cutoff, sehingga jauh lebih andal dipakai sebagai anti-aliasing filter analog sebelum ADC.

ANALOGI KEHIDUPAN SEHARI-HARI

Enam analogi berikut menghubungkan konsep filter & smoothing dengan pengalaman sehari-hari mahasiswa, mempermudah intuisi sebelum masuk ke rumus formal.

- **Noise-Cancelling Headphone:** sinyal antiphase dijumlahkan ke sinyal noise ambien sehingga saling meniadakan ($\text{music} = \text{signal} - \text{noise}$); analog dengan filter yang mengurangi komponen tak diinginkan dari sinyal campuran.
- **Kacamata Renang Anti-Embun:** lapisan anti-kabut pada kacamata renang bertindak seperti low-pass filter bertekstur, yaitu trade-off antara kejernihan pandang (passband) dan proteksi dari embun (attenuation).
- **Autofokus Kamera HP:** perpindahan fokus kamera HP antar objek berlangsung mulus, bukan instan; ini persis perilaku EMA: $\text{focus}_t = \alpha \cdot \text{target} + (1-\alpha) \cdot \text{focus}_{t-1}$.
- **Shutter Speed Eksposur:** durasi shutter terbuka meng-integrasi cahaya sepanjang periode tersebut, konsep yang identik dengan window Moving Average yang mengintegrasi/merata-ratakan sinyal sepanjang periode window.

- **Photoshop Gaussian vs Median Blur:** Gaussian blur meratakan piksel secara linear (mirip MA), sedangkan median blur mengganti tiap piksel dengan median tetangganya, sangat efektif menghapus noise 'salt-and-pepper' (bintik hitam-putih acak) tanpa mengaburkan tepi gambar.
- **Audacity Denoise Audio:** fitur Noise Reduction pada software audio memakai kombinasi spectral subtraction, median filter, dan smoothing mirip Savitzky-Golay, yaitu pipeline yang persis sama dengan preprocessing sinyal vibrasi di industri.

APLIKASI FILTER DI MESIN INDUSTRI

Pipeline pada Gambar 7 merangkum urutan filter yang direkomendasikan dalam praktik vibration monitoring: median filter dahulu untuk menghapus spike, lalu Savitzky-Golay untuk smoothing yang menjaga puncak, EMA untuk baseline yang halus, baru kemudian analisis (RMS/FFT/deteksi anomali).



Gambar 7. Pipeline filter rekomendasi untuk vibration monitoring: Raw Signal → Median Filter → Savitzky-Golay → EMA → Analisis.

- **Vibration RMS Baseline:** EMA dengan $\alpha \approx 0,05$ dipakai sebagai baseline bergerak RMS getaran ($EMA_t = 0,05 \cdot x_t + 0,95 \cdot EMA_{t-1}$); alert dipicu bila RMS melebihi $baseline + 3\sigma$, jauh lebih stabil dibanding threshold statis.
- **Spectral Peak Preservation:** Savitzky-Golay $w=11, p=3$ dipakai untuk identifikasi peak spektral bearing fault (BPFI/BPFO/BSF/FTF, lihat Pertemuan 7); MA akan menumpulkan peak spektral sehingga severity kerusakan bisa terestimasi lebih rendah dari kondisi sebenarnya.
- **Spike Removal:** median filter $w=5$ diterapkan sebelum perhitungan RMS untuk menghapus spike EMI/mechanical shock sesaat yang tidak merepresentasikan kondisi mesin sesungguhnya.
- **Anti-Aliasing Pre-Filter:** filter low-pass analog dengan cutoff = $f_s/2$ (Nyquist) wajib dipasang di hardware sebelum ADC. Untuk $f_s=10$ kHz, cutoff harus di sekitar 4 kHz agar komponen di atas Nyquist tidak menyebabkan aliasing.

Peringatan: Kesalahan praktis yang sering terjadi: (1) menerapkan smoothing time-domain sebelum FFT sehingga mendistorsi spektrum; (2) memakai MA panjang untuk deteksi shock/impact padahal tujuannya justru kontradiktif dengan sifat MA yang menumpulkan puncak; (3) lupa memasang anti-aliasing filter analog; (4) parameter α/k tidak disesuaikan dengan sampling rate sistem.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada data sintesis yang meniru sensor industri. Lingkungan: conda env optoauto dengan paket numpy, pandas, scipy, matplotlib; notebook p2_filter_smoothing.ipynb. Gambar 8 menunjukkan potongan Cell 1 yang membangkitkan sinyal sintesis dan menerapkan Moving Average.



```
import numpy as np

np.random.seed(42)
fs, T = 1000, 2.0
N = int(fs * T)
t = np.linspace(0, T, N, False)
clean = np.sin(2*np.pi*5*t)
noise = np.random.normal(0, 0.5, N)
spikes = np.zeros(N)
spikes[[100,500,1000,1500]] = 5
y = clean + noise + spikes

# Moving Average k=25
k = 25
ma = np.convolve(y, np.ones(k)/k,
                  mode='same')
print(f'RMS raw: {rms(y):.3f}')
print(f'RMS MA25: {rms(ma):.3f}')
```

Gambar 8. Potongan Cell 1: pembangkitan sinyal sintesis (noise + 4 spike) dan Moving Average $k=25$ dengan NumPy.

- **Cell 1:** generate sinyal noisy ($fs=1000$ Hz, 5 Hz sine + noise $\sigma=0,5$ + 4 spike di indeks [100,500,1000,1500]) + MA $k=5/25/100$, plot 4-panel, cetak RMS raw/clean/MA25.
- **Cell 2:** EMA manual (rekursif) vs `pandas.ewm()`, bandingkan $\alpha=[0,5; 0,1; 0,02]$ dengan overlay sinyal bersih sebagai ground truth.
- **Cell 3:** Savitzky-Golay vs Moving Average pada sinyal Gaussian peak ($N=500$), cetak persentase preservasi puncak (SG $\sim 95\%$, MA $< 70\%$).

- **Cell 4:** Median filter untuk penghapusan spike: tabel nilai pada 4 lokasi spike (Raw / MA k=5 / Median k=5 / Median+SG chained).

TUGAS PERTEMUAN 2

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Gambar 9 merangkum distribusi bobotnya.

Distribusi Bobot Tugas Pertemuan 2



Gambar 9. Distribusi 50 poin Tugas Pertemuan 2 berdasarkan tipe soal.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Tujuan utama dari filter low-pass pada data sensor adalah...

- A. Memperbesar amplitudo sinyal asli
- B. Mengurangi noise frekuensi tinggi sambil mempertahankan komponen frekuensi rendah
- C. Mengubah deret waktu menjadi domain frekuensi
- D. Menghitung autokorelasi sinyal

2. Pernyataan yang benar mengenai Trailing Moving Average (causal) adalah...

- A. Tidak memerlukan data masa lalu
- B. Cocok untuk real-time monitoring tapi memiliki lag $\approx (k-1)/2$ sampel
- C. Selalu menghasilkan zero phase shift
- D. Memerlukan polynomial fitting

3. Pada formula $EMAt = \alpha \cdot xt + (1-\alpha) \cdot EMAt-1$, jika $\alpha = 0,5$ maka...

- A. Output didominasi sepenuhnya oleh history (lag besar)
- B. Bobot setara antara observasi terbaru dan history, sangat responsif tapi noisy

- C. EMA tidak akan pernah mendekati sinyal asli
- D. α tidak boleh bernilai 0,5 secara matematis

4. Keunggulan utama Savitzky-Golay (SG) dibandingkan Moving Average untuk analisis getaran adalah...

- A. SG selalu lebih cepat secara komputasi
- B. SG menjaga amplitudo puncak/lembah jauh lebih baik (preserve features)
- C. SG tidak memerlukan parameter window
- D. SG bisa digunakan tanpa perlu polynomial order

5. Untuk menghapus impulse spike (lonjakan tunggal seperti EMI dari relay) dari sinyal sensor, filter yang paling efektif adalah...

- A. Moving Average dengan window besar
- B. EMA dengan α besar (responsif)
- C. Median filter, karena sifat nonlinear-nya menelan spike sepenuhnya
- D. Savitzky-Golay dengan polynomial order tinggi

6. Dampak utama dari memilih window MA yang terlalu besar pada deteksi anomali sinyal getaran adalah...

- A. Tidak ada dampak, sinyal tetap utuh
- B. Transient/lonjakan singkat ikut tersmoothing sehingga anomali terlewatkan, lag deteksi besar
- C. Konsumsi memori turun drastis
- D. Output menjadi sama dengan EMA dengan $\alpha=1$

7. Hubungan antara α EMA dan effective window N equivalent adalah...

- A. $N = \alpha \times 100$
- B. $\alpha = 2/(N+1)$, atau ekuivalen $N = 2/\alpha - 1$
- C. $N = \log(\alpha)$
- D. Tidak ada hubungan formal

8. Yang bukan merupakan sumber noise pada sensor industri adalah...

- A. Thermal noise pada resistor
- B. Quantization noise dari ADC resolusi terbatas
- C. EMI dari motor/inverter di lingkungan
- D. Bias DC dari kalibrasi (ini systematic offset, bukan noise)

9. Pipeline pre-processing standar untuk vibration data dengan noise Gaussian + spike adalah...

- A. FFT terlebih dahulu, lalu MA, lalu SG
- B. Median (hapus spike), lalu Savitzky-Golay (preserve peak smoothing), lalu analisis
- C. EMA dengan α besar, lalu MA window besar, lalu median

- D. Tidak ada urutan baku, tergantung intuisi

10. Mengapa tidak disarankan menerapkan smoothing time-domain (MA/EMA) sebelum FFT pada analisis getaran?

- A. FFT tidak akan berjalan pada sinyal yang di-filter
- B. Filter mendistorsi spektrum, peak amplitudo dan posisi frekuensi tidak akurat sehingga severity assessment salah
- C. Smoothing meningkatkan noise di domain frekuensi
- D. Operasi Python tidak mendukung pipeline tersebut

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Dataset referensi (dipakai C₁-C₁₀, dibuat sekali di Jupyter):

```
import numpy as np
np.random.seed(42)
fs = 1000; T_dur = 2.0; N = int(fs*T_dur)
t = np.linspace(0, T_dur, N, endpoint=False)
clean = np.sin(2*np.pi*5*t)           # 5 Hz pure baseline
noise = np.random.normal(0, 0.5, N)   # heavy gaussian noise
spikes = np.zeros(N); spikes[[100,500,1000,1500]] = 5
y = clean + noise + spikes             # deret yang akan difilter
```

C1. Hitung RMS dari sinyal y mentah (raw, sebelum filter). Bandingkan dengan RMS clean signal ($\approx 0,707$); selisihnya menunjukkan kontribusi noise + spike.

-  **HINT:** RMS = akar dari rata-rata kuadrat sinyal; gunakan np.sqrt dan np.mean pada y^{**2} .

C2. Terapkan Moving Average window k=10 memakai np.convolve(y, np.ones(10)/10, mode='same'). Cetak mean dari hasil filter.

-  **HINT:** Pakai np.convolve dengan kernel np.ones(k)/k dan mode='same', lalu ambil rata-rata hasilnya dengan np.mean.

C3. Terapkan Moving Average window k=50. Cetak standar deviasi (ddof=1) dari hasil filter; std akan jauh lebih kecil dari raw karena noise tersmoothing.

-  **HINT:** Terapkan moving average dengan np.convolve (kernel np.ones(k)/k), lalu hitung std sampel dengan np.std(..., ddof=1).

C4. Terapkan EMA dengan $\alpha=0,1$ memakai pandas: ema = pd.Series(y).ewm($\alpha=0.1$, adjust=False).mean().values. Cetak nilai EMA pada indeks terakhir.

-  **HINT:** Gunakan pd.Series(y).ewm($\alpha=...$, adjust=False).mean(), lalu ambil nilai pada indeks terakhir.

C5. Terapkan EMA dengan $\alpha=0,5$ (sangat responsif). Cetak nilai EMA pada indeks 1500 (segera setelah spike ke-4). EMA dengan α besar akan menyerap spike dengan kuat.

-  **HINT:** Hitung EMA responsif dengan ewm($\alpha=...$, adjust=False).mean(), lalu ambil nilai pada indeks yang diminta soal.

C6. Terapkan Savitzky-Golay (window=11, polyorder=3) dari `scipy.signal`. Cetak nilai filtered pada indeks 1000 (lokasi spike).

- 💡 **HINT:** Gunakan `scipy.signal.savgol_filter` dengan window dan polyorder dari soal, lalu ambil nilai pada indeks yang diminta.

C7. Terapkan Median filter dengan `kernel_size=5` dari `scipy.signal`. Cetak nilai pada indeks 100 (lokasi spike pertama). Median filter akan sepenuhnya menghapus spike.

- 💡 **HINT:** Gunakan `scipy.signal.medfilt` dengan `kernel_size` dari soal, lalu ambil nilai pada indeks spike yang diminta.

C8. Hitung effective window N equivalent untuk EMA dengan $\alpha=0,1$. Rumus: $N = 2/\alpha - 1$. Cetak nilai N .

- 💡 **HINT:** Effective window $N = 2/\alpha - 1$. Substitusikan nilai α dari soal.

C9. Hitung RMS dari residual (selisih antara y dengan clean): $\text{residual} = y - \text{clean}$. Cetak RMS residual, yang mengukur kontaminasi noise + spike total.

- 💡 **HINT:** Hitung $\text{residual} = y - \text{clean}$, lalu RMS-nya dengan `np.sqrt(np.mean(residual**2))`.

C10. Terapkan MA $k=25$ pada y lalu hitung RMS hasilnya. Bandingkan dengan RMS clean (0,707); MA window 25 cukup baik untuk recovery RMS asli.

- 💡 **HINT:** Terapkan moving average $k=25$ dengan `np.convolve`, lalu hitung RMS hasilnya dengan `np.sqrt(np.mean(...))`.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal (bukan memanggil fungsi library siap pakai untuk bagian intinya), 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Implementasikan Moving Average dari awal tanpa `np.convolve` atau `pandas`. Untuk window $k=15$ dan dataset referensi y , gunakan loop: untuk tiap titik i , hitung mean dari $y[\max(0, i-7):\min(N, i+8)]$ (centered, dengan boundary handling). Cetak mean dari MA hasil.

- 💡 **HINT:** Loop tiap titik i , ambil window centered $y[\max(0, i-7):\min(N, i+8)]$ dan hitung mean-nya untuk mengisi `out[i]`; akhirnya ambil rata-rata seluruh `out`.

C12 (Hard). Implementasikan EMA dari awal tanpa `pandas`. Untuk dataset y dengan $\alpha=0,05$ dan inisialisasi $\text{EMA}[0] = \text{mean}(y[:10])$ (bukan default $y[0]$). Cetak nilai EMA pada indeks 100.

-  **HINT:** Inisialisasi $EMA[0] = \text{mean dari 10 titik pertama}$, lalu iterasi rumus rekursif $EMA_i = \alpha \cdot y_i + (1-\alpha) \cdot EMA_{i-1}$; ambil nilai pada indeks yang diminta.

C13 (Hard). Implementasikan Savitzky-Golay dari awal tanpa `scipy.signal.savgol_filter`. Untuk `window=11`, `polyorder=3`, dan dataset `y`: untuk tiap pusat window, fit polinomial orde 3 dengan `np.polyfit(idx, y[i-5:i+6], 3)`, lalu evaluasi di pusat (`idx=0`). Cetak nilai SG manual pada indeks 500.

-  **HINT:** Untuk tiap pusat window, fit polinomial orde 3 dengan `np.polyfit` pada 11 titik sekitar, lalu evaluasi di pusat (`idx=0`) memakai `np.polyval`; ambil nilai pada indeks yang diminta dan validasi terhadap `savgol_filter`.

C14 (Hard). Implementasikan anomaly detection berbasis EMA + aturan 3σ . Generate sinyal baru: $y_{anom} = \text{clean} + \text{noise}$ (tanpa spike), lalu inject 3 anomali besar: $y_{anom}[[250, 750, 1250]] += 4.0$. Hitung $EMA(\alpha=0,05)$ lalu $\text{residual} = y_{anom} - EMA$. Hitung $n_{detect} = \text{jumlah } |\text{residual}| > 3 \cdot \text{std}(\text{residual})$. Cetak jumlah anomali terdeteksi.

-  **HINT:** Bangun sinyal beranomali sesuai soal, hitung EMA-nya, lalu $\text{residual} = \text{sinyal} - EMA$; hitung berapa titik dengan $|\text{residual}|$ melebihi $3 \times \text{std residual}$.

C15 (Hard). Implementasikan filter pipeline lengkap pada `y`: (1) median filter $k=5$ untuk hapus spike, (2) Savitzky-Golay $w=11, p=3$ untuk smoothing yang preserve peak, (3) EMA $\alpha=0,1$ untuk baseline yang halus. Hitung RMS dari output akhir pipeline. Hasil seharusnya mendekati RMS clean ($\approx 0,707$).

-  **HINT:** Rangkai tiga tahap berurutan: `medfilt` $\rightarrow$ `savgol_filter` $\rightarrow$ `ewm().mean()` sesuai parameter soal, lalu hitung RMS output akhir dengan `np.sqrt(np.mean)`.

DAFTAR PUSTAKA

- Dudek, G. (2023). STD: A seasonal-trend-dispersion decomposition of time series. IEEE Transactions on Knowledge and Data Engineering, 35(10), 10339–10350. <https://doi.org/10.1109/TKDE.2023.3268125>
- Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- Khan, S., & Lee, D.-H. (2017). An adaptive dynamically weighted median filter for impulse noise removal. EURASIP Journal on Advances in Signal Processing, 2017, 67. <https://doi.org/10.1186/s13634-017-0502-z>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. Eng, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>

Schmid, M., Rath, D., & Diebold, U. (2022). Why and how Savitzky-Golay filters should be replaced. *ACS Measurement Science Au*, 2(2), 185–196.

<https://doi.org/10.1021/acsmeasuresciau.1c00054>

Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>